

UNIVERSIDADE FEDERAL DE ALFENAS
CURSO DE CIÊNCIAS DA COMPUTAÇÃO

TRABALHO PRÁTICO: IMPLEMENTAÇÃO DO PROTOCOLO GO-BACK-N SOBRE UDP

RELATÓRIO TÉCNICO E ANÁLISE DE DESEMPENHO

Grupo:

Eurico Santiago Climaco Rodrigues
Diogo Peixoto Rossini

Disciplina: Redes de Computadores

Professor: Dr. Flávio Barbieri Gonzaga

Alfenas - MG
Junho de 2026

Conteúdo

1	Introdução	2
2	Arquitetura e Implementação	2
2.1	Formato do Segmento (Cabeçalho)	2
2.2	Handshake e Estabelecimento da Conexão	2
2.3	Máquina de Estados e Lógica do Emissor	3
2.4	Máquina de Estados e Lógica do Receptor	3
2.5	Encerramento da Conexão (FIN)	4
3	Metodologia e Resultados Experimentais	4
3.1	Resultados: Experimento 1 (Variação de N)	4
3.1.1	Análise da Variação de N	5
3.2	Resultados: Experimento 2 (Variação de p)	5
3.2.1	Análise da Variação de p	6
4	Dificuldades Enfrentadas e Soluções	7
5	Conclusão	7

1 Introdução

O objetivo deste trabalho prático consiste na implementação e análise experimental de um protocolo de transferência confiável de dados baseado no algoritmo *Go-Back-N* (GBN). O protocolo foi construído diretamente sobre o protocolo de transporte UDP (que não oferece confiabilidade nativa), utilizando a linguagem de programação Java.

O protocolo GBN pertence à categoria de protocolos de janela deslizante (*sliding window*), projetados para otimizar o uso do canal de comunicação ao permitir o envio de múltiplos pacotes antes de aguardar por suas respectivas confirmações (ACKs). O controle de fluxo e a recuperação de erros são mantidos através de uma janela de tamanho limitado N , temporizadores de retransmissão no emissor e a adoção de ACKs cumulativos pelo receptor.

Este relatório detalha as decisões de projeto adotadas na modelagem do formato de pacotes (segmentos), as máquinas de estados do emissor e receptor, as dificuldades técnicas enfrentadas no desenvolvimento e a avaliação de desempenho baseada em variações controladas do tamanho da janela N e da probabilidade de perda de pacotes p .

2 Arquitetura e Implementação

A arquitetura do sistema foi estruturada em dois módulos independentes construídos em Java: o **Emissor** (Sender) e o **Receptor** (Receiver). A comunicação ocorre via *DatagramSockets* do Java operando sobre UDP.

2.1 Formato do Segmento (Cabeçalho)

Para garantir a modularidade e o empacotamento correto dos dados, foi definida uma estrutura de dados de segmento customizada que encapsula o cabeçalho e a carga útil. A classe **Segmento** realiza a serialização dos campos em um array de bytes com tamanho fixo de 1024 bytes. O formato do segmento possui os seguintes campos:

Campo	Tipo	Tamanho (Bytes)	Descrição
Tipo	Byte	1	0 = DATA, 1 = ACK, 2 = HANDSHAKE, 3 = FIN
Num_Seq	Int	4	Número de sequência do pacote enviado
Num_Ack	Int	4	Número de confirmação cumulativa (ACK)
Tamanho_Dados	Short	2	Quantidade de bytes válidos no payload
Dados	Byte[]	Até 1013	Dados úteis lidos do arquivo de origem

Figura 1: Estrutura do Cabeçalho e Payload do Segmento Customizado.

A serialização é realizada utilizando a biblioteca **ByteBuffer** do Java, garantindo a consistência na ordem dos bytes (*byte ordering*) e prevenindo falhas de interpretação de dados entre processos. A carga útil máxima por segmento de dados é de $1024 - 11 = 1013$ bytes.

2.2 Handshake e Estabelecimento da Conexão

Antes de iniciar a transferência de dados, é realizada uma fase de handshake de três vias simplificada:

1. O **Emissor** envia um segmento do tipo **HANDSHAKE** contendo uma carga útil formatada em string contendo: a probabilidade de perda nominal p a ser simulada no receptor, o tamanho total do arquivo de origem (em bytes) e o caminho completo onde o arquivo de destino deve ser salvo no receptor.
2. O **Receptor** recebe este segmento, extrai os parâmetros, prepara a infraestrutura para a gravação física do arquivo local e envia de volta um segmento do tipo **ACK** com o campo `num_ack` = -1.
3. Ao receber a confirmação, o Emissor altera seu estado interno para conectado e inicia a fase de transferência de dados. O timeout do socket do Emissor para esta fase é configurado em 1.000 ms, ocorrendo até 10 retransmissões do handshake em caso de perda.

2.3 Máquina de Estados e Lógica do Emissor

A lógica do Emissor baseia-se na máquina de estados tradicional do *Go-Back-N* descrita por Kurose e Ross. A sua execução baseia-se em um loop não bloqueante principal, estruturado da seguinte forma:

- **Envio dentro da Janela:** O Emissor mantém duas variáveis de controle de sequência: `base` (o número de sequência do pacote mais antigo não confirmado) e `nextseqnum` (o próximo número de sequência a ser enviado). O Emissor transmite novos pacotes contanto que `nextseqnum < base + N`.
- **Gerenciamento de ACKs:** O Emissor monitora o recebimento de ACKs do receptor chamando `socket.receive()` com um timeout curto (10 ms) em um bloco try-catch para implementar comportamento não bloqueante de forma síncrona. Quando um ACK é recebido com `num_ack >= base`, a janela é deslocada definindo-se `base = num_ack + 1`. O temporizador é reiniciado se houver pacotes não confirmados na janela, ou desativado caso a janela fique vazia.
- **Tratamento de Timeout:** O Emissor mantém o registro do tempo em que o pacote mais antigo em trânsito (`base`) foi enviado. Se a diferença de tempo em milissegundos exceder o limite fixado de 300 ms, ocorre a expiração (*Timeout*). O Emissor então retransmite **todos** os pacotes da janela corrente entre `base` e `nextseqnum - 1` e reinicia o temporizador.

2.4 Máquina de Estados e Lógica do Receptor

O Receptor implementa uma lógica simplificada e robusta:

- Mantém a variável `expectedseqnum` inicializada em 0.
- Para cada pacote de dados recebido:
 1. Simula uma perda pseudo-aleatória: gera-se um número real r no intervalo $[0.0, 1.0)$. Se $r < p$, o pacote é descartado imediatamente, simulando-se perda na rede.
 2. Se o pacote não for perdido, verifica-se se o número de sequência do pacote (`seqNum`) é exatamente igual a `expectedseqnum`.

3. **Se for igual (em ordem):** O Receptor grava os dados no arquivo de saída, envia um ACK contendo o número do pacote recebido (`num_ack = expectedseqnum`), incrementa `expectedseqnum` em 1 e atualiza o contador de pacotes recebidos com sucesso.
4. **Se for diferente (fora de ordem):** O Receptor descarta o pacote e reenvia um ACK cumulativo para o último pacote recebido em ordem (`num_ack = expectedseqnum - 1`).

2.5 Encerramento da Conexão (FIN)

Após a confirmação bem-sucedida de todos os pacotes de dados (`base == totalPacotes`), o Emissor envia um pacote especial do tipo FIN (tipo 3) para indicar o encerramento da transmissão. O Receptor, ao receber o FIN, fecha o fluxo de gravação física do arquivo, calcula o hash MD5 da cópia recebida, exibe as estatísticas locais de transmissão e encerra sua execução após responder com um ACK para o FIN.

3 Metodologia e Resultados Experimentais

Para avaliar o desempenho prático do protocolo implementado, foi desenvolvido um script de automação em Python. O script compila os projetos com Maven, gera arquivos binários de tamanho conhecido (100 KB) e executa instâncias locais dos programas simulando cenários variados.

Foram desenhados dois experimentos principais:

1. **Experimento 1 (Variando N):** Tamanho da janela $N \in \{1, 2, 4, 8, 16, 32\}$ com probabilidade de perda nominal fixa em $p = 10\%$.
2. **Experimento 2 (Variando p):** Probabilidade de perda nominal $p \in \{0\%, 2\%, 5\%, 10\%, 20\%, 30\%\}$ com tamanho de janela fixo em $N = 8$.

3.1 Resultados: Experimento 1 (Variação de N)

A Tabela 1 exibe os dados coletados ao variar a janela de transmissão com perda fixa em 10%.

N	Tempo (s)	Throughput (KB/s)	Pacotes Originais	Retransmissões	Perda Efetiva
1	4.814	77.00	102	12	10.53%
2	4.523	11.00	102	26	11.30%
4	3.640	47.00	102	44	9.73%
8	3.783	43.00	102	96	10.53%
16	4.312	19.00	102	216	12.07%
32	5.553	1.00	102	524	15.00%

Tabela 1: Resultados experimentais variando o tamanho da janela N ($p = 10\%$).

A Figura 2 apresenta a representação gráfica do impacto do tamanho da janela no tempo de transferência e vazão.

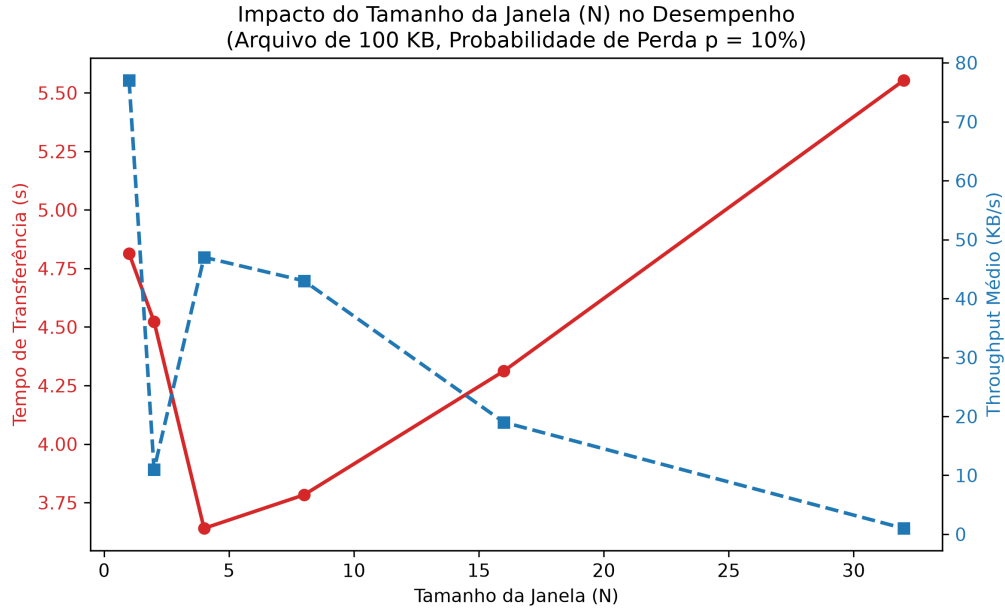


Figura 2: Gráfico comparativo de tempo e vazão em função do tamanho da janela N ($p = 10\%$).

3.1.1 Análise da Variação de N

Os dados mostram um comportamento clássico do protocolo Go-Back-N.

- Para $N = 1$ (equivalente ao protocolo *Stop-and-Wait*), o emissor envia um pacote e espera sua confirmação. A vazão é limitada devido ao tempo de ida e volta da rede (RTT), mas o número de retransmissões é proporcional à perda real da rede (12 retransmissões para 102 pacotes originais, o que condiz com os 10% nominais).
- Conforme N aumenta para 2, 4 e 8, há uma melhora no aproveitamento do link, permitindo que a transmissão termine mais rápido (o tempo cai de 4.814 s para 3.640 s com $N = 4$).
- Contudo, ao expandirmos a janela para $N = 16$ e $N = 32$, o tempo de transmissão volta a subir (5.553 s com $N = 32$) e o número de retransmissões explode para 524 pacotes. Isso ocorre porque o GBN descarta pacotes fora de ordem no receptor. Em uma janela de tamanho 32, a perda de um único pacote no início da janela faz com que todos os outros 31 pacotes enviados em seguida sejam descartados pelo receptor. O emissor retransmite a janela inteira após o timeout de 300 ms. Se a perda persistir nas retransmissões, o link fica congestionado de retransmissões redundantes, degradando severamente o desempenho.

3.2 Resultados: Experimento 2 (Variação de p)

A Tabela 2 exibe os dados experimentais coletados ao variar a probabilidade de perda nominal de pacotes com tamanho de janela fixo em $N = 8$.

p	Tempo (s)	Throughput (KB/s)	Pacotes Originais	Retransmissões	Perda Efeti
0%	0.189	10.00	102	0	0.00%
2%	1.383	31.00	102	32	3.77%
5%	1.688	24.00	102	38	4.67%
10%	7.383	54.00	102	182	19.05%
20%	7.420	48.00	102	181	19.05%
30%	13.733	28.00	102	357	30.61%

Tabela 2: Resultados experimentais variando a probabilidade de perda nominal p ($N = 8$).

A Figura 3 apresenta graficamente o impacto da perda no tempo de transferência e quantidade de retransmissões.

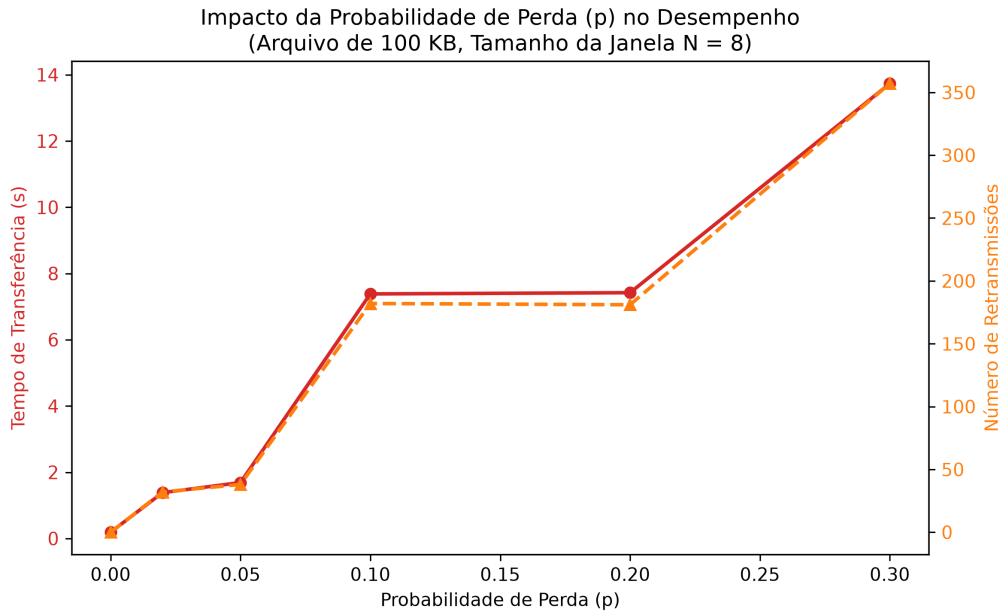


Figura 3: Gráfico comparativo de tempo e retransmissões em função da taxa de perda p ($N = 8$).

3.2.1 Análise da Variação de p

A taxa de perda possui impacto direto e agressivo no Go-Back-N.

- Com $p = 0\%$, não há perdas. A transmissão ocorre no melhor caso teórico do canal UDP local, completando a cópia dos 100 KB em 0.189 segundos, sem nenhuma retransmissão.
- Para perdas moderadas de 2% e 5%, ocorrem poucas falhas e o tempo permanece baixo (menos de 1.7 segundos). A perda efetiva medida reflete de perto a probabilidade nominal configurada.
- A partir de 10%, o cenário muda drasticamente: o tempo salta para 7.383 segundos e as retransmissões disparam para 182. A 30% de perda, o tempo atinge 13.733 segundos e ocorrem 357 retransmissões redundantes. Cada timeout impõe um atraso

ocioso de 300 ms no emissor, e como a janela é relativamente grande ($N = 8$), retransmite-se muito lixo para cada perda recuperada. A eficiência de vazão cai e o tempo aumenta linearmente em relação ao número total de timeouts acumulados.

4 Dificuldades Enfrentadas e Soluções

Durante a fase de desenvolvimento prático e teste do protocolo, o grupo deparou-se com três desafios centrais:

1. **Leitura Não Bloqueante no DatagramSocket:** Por padrão, a chamada `socket.receive()` em Java bloqueia a execução da thread corrente até que um pacote chegue. Isso impedia o emissor de gerenciar o temporizador da janela deslizante e enviar novos pacotes simultaneamente. A solução clássica envolveria multithreading complexo com locks de sincronização para proteger as variáveis compartilhadas (`base` e `nextseqnum`). Como alternativa mais simples e robusta, optou-se por configurar o timeout do socket no emissor para um valor muito baixo (`socket.setSoTimeout(10)`). No loop de envio, tentava-se ler o ACK de forma síncrona, mas capturando a exceção `SocketTimeoutException` para continuar imediatamente o loop e processar o temporizador de timeout geral de 300 ms.
2. **Formatação Regional em Strings (Bug de Locale):** Durante a automação dos testes experimentais, os valores de taxa de perda efetiva extraídos do Receptor pelo script Python eram bizarramente interpretados. Por exemplo, uma taxa real de 0.97% era lida pelo Python como 97%. Descobriu-se que, devido ao locale padrão da máquina local configurado para português do Brasil, o método Java `System.out.printf()` exibia os decimais usando a vírgula como separador (0,97%). A expressão regular utilizada no script Python (`[\d.]+%`) não correspondia à vírgula, fazendo o casamento apenas sobre os algarismos à direita do separador. A correção foi implementada ajustando a expressão regular no script Python para `[\d.,]+%` e substituindo caracteres de vírgula por ponto antes da conversão para float no script.
3. **Ajuste Fino do Timeout da Janela:** Inicialmente, utilizou-se um temporizador de 100 ms no Emissor. Contudo, em localhost, devido à concorrência de execução de processos Java e o receptor simulando atrasos de escrita física, ocorriam timeouts prematuros (*spurious timeouts*), gerando retransmissões de janelas inteiras antes que os ACKs tivessem tempo físico de voltar ao socket. Aumentando o timeout para 300 ms, o comportamento estabilizou e evitou-se retransmissões desnecessárias.

5 Conclusão

A implementação prática do protocolo Go-Back-N sobre UDP permitiu a consolidação empírica de conceitos fundamentais da camada de transporte estudados teoricamente.

O trabalho demonstrou que, embora o GBN melhore consideravelmente a eficiência da transmissão em redes com baixo RTT e taxas de erro pequenas através do paralelismo oferecido pela janela deslizante, ele apresenta sérias limitações sob taxas de perda moderadas a altas ($p \geq 10\%$). A política do receptor de descartar pacotes fora de ordem gera um excesso de tráfego redundante (retransmissão da janela inteira) que satura os recursos

da rede. Sob estas condições adversas, o protocolo *Selective Repeat* seria uma alternativa preferível devido ao seu mecanismo de buffering e retransmissão seletiva, reduzindo o tráfego redundante observado neste experimento.

A integridade de todos os arquivos transmitidos foi verificada por meio do cálculo de hashes MD5 tanto na origem quanto no destino, confirmando o sucesso da implementação do protocolo de transporte confiável de dados do grupo.